YOLO26n-Based Bowling Gameplay Analysis and Scoring System

Model Training, Evaluation, and Bowling Gameplay Inference Report

Abdelrahman Abdelrassoul - 202300056

Ali Ashraf - 202301812

Submission Date: May 8, 2026

Contents

1 Abstract **1**

2 Introduction **1**

3 Problem Statement **1**

4 Objectives **2**

5 Dataset Description **2**
5.1 Data Source 2
5.2 Dataset Configuration 3
5.3 Dataset Statistics 3

6 Data Exploration and Analysis **3**
6.1 File Structure Inspection 3
6.2 Class Name Verification 3
6.3 Annotation Visualization 4

7 Data Cleaning **4**

8 Missing Values Handling **5**

9 Feature Engineering **5**

10 Feature Selection **6**

11 Data Preprocessing **6**
11.1 Training Preprocessing 6
11.2 Inference Preprocessing 6

12 Data Augmentation **6**

13 Data Splitting **7**

14 Model Architecture **7**
14.1 YOLO26n Overview 7
14.2 Architecture Blocks 7
14.3 Input and Output Shapes 8

15 Model Design Decisions **8**

16 Training Pipeline **8**
16.1 Hyperparameters 9

17 Loss Functions **9**

18 Optimizer and Learning Rate Strategy **9**

19 Training Procedure **10**

20 Validation Logic **14**

21 Evaluation Metrics **14**

22 Experimental Results **15**
22.1 Final Validation Results 15
22.2 Held-Out Test Results 15
22.3 Inference Speed 16

23 Testing Pipeline **16**

24 Model Checkpoint Selection and Benchmarking **17**
24.1 Benchmark Results 17

25 Video Inference Pipeline **18**
25.1 Model Loading 18
25.2 Video Processor 18
25.3 Video Data Flow Diagram 19

26 Prediction Logic **19**

27 Tracking Logic **20**
27.1 PinTrack Class 20
27.2 Matching Strategy 20
27.3 Track Creation and Removal 21
27.4 Fall Confirmation 21

28 Overlay Rendering **21**

29 Metrics and Output Files from Video Testing **21**

30 Error Analysis **22**

31 Performance Analysis **22**

32 Challenges Faced **22**

33 Future Improvements **23**

34 Conclusion **23**
The developed model was later adapted for use in another separate application outside the scope of this report.

List of Figures

1 Dataset annotation visualization for bowling gameplay frames showing bowling ball and pin bounding boxes. 4
2 Training and validation loss curves over epochs for the bowling gameplay analysis model. 10
3 Validation precision, recall, and F1-related performance trends across training epochs. 11
4 Validation mAP@0.50 and mAP@0.50:0.95 curves for bowling ball and pin detection. 11
5 Normalized Confusion matrix for bowling gameplay object detection and pin-state classification. 12
6 Precision–recall curve for bowling ball and pin detection performance. . . 12
7 F1-confidence curve showing the effect of confidence threshold selection on bowling gameplay detection performance. 13
8 Precision-confidence curve showing precision variation across confidence thresholds for bowling gameplay detection. 13
9 Recall-confidence curve showing recall variation across confidence thresholds for bowling gameplay detection. 14
10 YOLO26n prediction results on bowling gameplay frames showing detected bowling balls and pins. 17
11 Video inference and scoring pipeline implemented in the testing notebook. 19

List of Tables

1 Dataset annotation counts extracted from the training notebook. 3
2 Main training hyperparameters used in the notebook. 9
3 Final validation metrics for the best YOLO26n checkpoint. 15
4 Held-out test metrics reported by the notebook. 15
5 Validation speed reported by Ultralytics. 16
6 Selected model benchmark results reported by the notebook. 17
7 Class-specific thresholds in the video testing notebook. 20

1 Abstract

This report documents the complete machine learning and computer vision workflow implemented across two notebooks: `cv-project-train.ipynb` and `cv-bonus-test.ipynb`. The project builds a custom YOLO26n object detector for an bowling-bowling scene and evaluates it through both image-level and video-level experiments. The training notebook downloads a Roboflow bowling gameplay dataset, verifies the four target classes, normalizes the bowling gameplay dataset configuration, explores labels, visualizes annotations, trains a YOLO26n detector through transfer learning, evaluates the model on validation and test splits, and runs sample predictions. The testing notebook implements a video-level inference and scoring pipeline that loads trained model files, processes a user-provided video, performs frame-wise detection, tracks pin identities, assigns fall-order labels, draws bowling trajectory paths, saves annotated videos, and writes metrics summaries.

The final trained model achieved strong validation performance with precision of 0.909, recall of 0.880, mAP@0.50 of 0.915, and mAP@0.50:0.95 of 0.744 on the validation split. On the held-out test split, the detector achieved an overall precision of 0.780, recall of 0.745, mAP@0.50 of 0.762, and mAP@0.50:0.95 of 0.444. These results show that the detector learned the task effectively, especially for standing pins, while fallen pins remained more challenging due to occlusion, motion blur, object overlap, and variable orientations. The complete implementation combines lightweight object detection with deterministic temporal tracking to produce a practical video scoring pipeline.

2 Introduction

The objective of this project is to detect and analyze events in a toy bowling-bowling setup. In this setup, a small remote-controlled bowling collides with bowling pins while a camera records the interaction. The computer vision system must detect objects in the scene, distinguish standing pins from fallen pins, track objects across frames, determine the order in which pins fall, and generate an annotated output video.

The project is divided into two major technical workflows. The first workflow is the model training and evaluation pipeline implemented in `cv-project-train.ipynb`. This notebook focuses on bowling gameplay dataset loading, validation, preprocessing, YOLO26n transfer learning, evaluation, and prediction visualization. The second workflow is the bowling gameplay inference and scoring pipeline implemented in `cv-bonus-test.ipynb`. This notebook focuses on applying trained models to full videos rather than still images, maintaining pin identities over time, estimating bowling trajectory, and exporting annotated videos.

This report is written so that the complete implementation can be understood without opening the notebooks. It explains the logic of each major code block, the motivation behind design decisions, the exact metrics extracted from the notebook outputs, and the relationship between detection results and the final scoring logic.

3 Problem Statement

The problem is a multi-object detection and temporal event analysis task. Given a recorded video of an bowling interacting with bowling pins, the system must identify the

relevant objects in each frame and infer higher-level events from frame sequences.

The task is challenging because:

- pins are small and can overlap with each other;
- fallen pins can appear in many different orientations;
- the bowling moves quickly and can introduce motion blur;
- the camera may be handheld, causing frame-to-frame motion;
- frame-wise detections alone do not identify which physical pin is which;
- duplicate detections may appear on the same object;
- a scoring system requires temporal reasoning, not only object detection.

Therefore, the final solution requires both a trained object detector and a temporal tracking layer.

4 Objectives

The project objectives are:

1. collect and use a labeled bowling gameplay dataset for bowling-bowling scenes;
2. train a lightweight YOLO26n object detector for four classes: ball, bowling, fallen-pins, and standing-pins;
3. evaluate the detector on validation and test data;
4. implement a bowling gameplay bowling gameplay analysis pipeline capable of processing complete videos;
5. track pins across frames using geometric matching;
6. detect standing-to-fallen transitions and assign fall-order labels;
7. draw bounding boxes, fall numbers, timing information, and bowling trajectory overlays;
8. save annotated output videos and model comparison metrics.

5 Dataset Description

5.1 Data Source

The training notebook downloads the bowling gameplay dataset from Roboflow The exported bowling gameplay dataset is saved under the Kaggle working directory. The bowling gameplay dataset metadata indicates four classes:

- `ball`
- `bowling`
- `fallen-pins`
- `standing-pins`

The notebook verifies these class names explicitly using an assertion. This is important because an incorrect class order would break downstream tracking logic, where class IDs are assumed to map to fixed semantic labels.

5.2 Dataset Configuration

The original Roboflow `data.yaml` initially contains relative paths. The notebook rewrites it to match the Kaggle runtime directory:

```
data_yaml["path"] = DATASET_DIR
data_yaml["train"] = "train/images"
data_yaml["val"] = "valid/images"
data_yaml["test"] = "test/images"
```

This ensures that Ultralytics can locate the training, validation, and test folders correctly.

5.3 Dataset Statistics

The notebook counts all labels by reading every YOLO label file and accumulating class IDs. Table 1 summarizes the extracted counts.

Table 1: Dataset annotation counts extracted from the training notebook.

Split	ball	bowling	fallen-pins	standing-pins	Total
Train	2323	1828	8327	16681	29159
Validation	210	177	896	1762	3045
Test	112	78	404	742	1336

The class distribution is imbalanced. Standing pins dominate the bowling gameplay dataset, followed by fallen pins. The bowling and ball classes are less frequent. This imbalance explains why standing pins achieved the strongest final performance, while ball and fallen-pin detection were comparatively more difficult.

6 Data Exploration and Analysis

6.1 File Structure Inspection

The notebook uses a shell command to inspect the downloaded bowling gameplay dataset:

```
find $DATASET_DIR -maxdepth 3 -type f | head -50
```

This verifies that the bowling gameplay dataset contains the expected subdirectories, including image folders and label folders for train, validation, and test splits.

6.2 Class Name Verification

The notebook prints and validates the class list:

```
expected_classes = ["ball", "bowling", "fallen-pins", "standing-pins"]
assert data_yaml.get("names") == expected_classes or \
      set(data_yaml.get("names", [])) == set(expected_classes)
```

This step prevents mismatched class interpretation during training and inference.

6.3 Annotation Visualization

The function `draw_yolo_labels` reads YOLO-format normalized labels, converts them into pixel-space bounding boxes, and draws them over random bowling gameplay frames. This allows visual confirmation that:

- boxes align with the objects;
- class IDs correspond to the correct labels;
- annotations are suitable for object detection training.

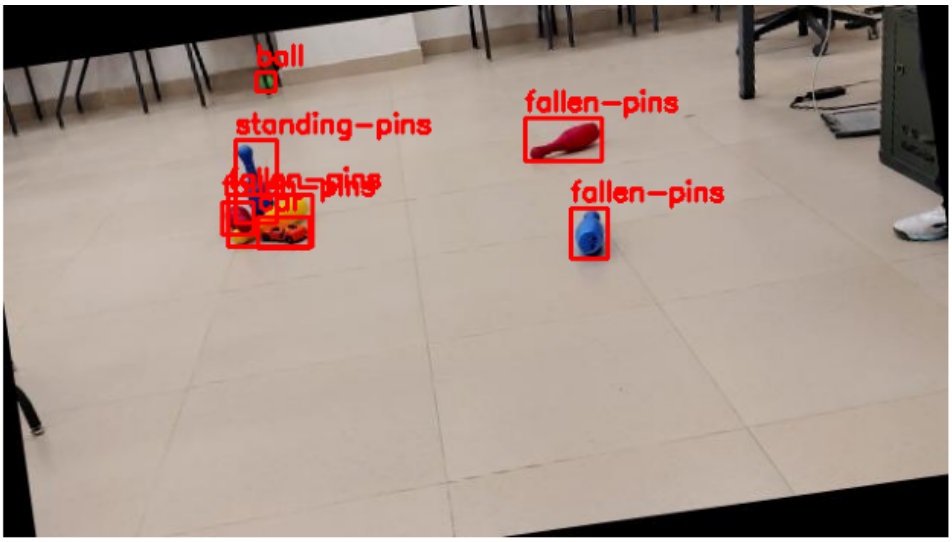

Figure 1: Dataset annotation visualization for bowling gameplay frames showing bowling ball and pin bounding boxes.

7 Data Cleaning

The bowling gameplay dataset was already exported in YOLO format by Roboflow, but the notebook and Ultralytics training logs performed several cleaning and validation operations:

- missing image-label pairs were avoided by relying on Roboflow's split structure;
- duplicate labels were automatically removed by Ultralytics during bowling gameplay dataset scanning;
- mixed box/segment information was resolved by using bounding boxes only;
- bowling gameplay dataset paths were rewritten in `data.yaml` for the active runtime.

The training output reported multiple duplicate labels removed from several images. This indicates that the training engine detected overlapping or repeated annotation entries and cleaned them before optimization. It also reported that box and segment counts were not equal, so segmentation data were discarded and only detection boxes were used. This is appropriate because the project is an object detection task rather than an instance segmentation task.

8 Missing Values Handling

The notebooks did not process tabular data, so traditional missing-value imputation was not applicable. Instead, missingness was handled at the bowling gameplay dataset file level:

- the bowling gameplay dataset folder was verified to exist;
- image and label folders were checked through directory listing;
- labels were read only when corresponding files existed;
- Ultralytics bowling gameplay dataset scanning confirmed zero corrupt images in the train, validation, and test splits.

The relevant missing-data concept for this project is missing detections during bowling gameplay inference. The testing notebook handles this by giving each pin track a `missing_frames` counter and keeping tracks alive for a limited number of frames before removal.

9 Feature Engineering

For YOLO model training, no handcrafted image features are created. The deep learning model learns hierarchical visual features directly from RGB image data. However, the video-level inference notebook performs geometric feature engineering for tracking and scoring. These derived features include:

- bounding-box center coordinates;
- bounding-box area;
- IoU between boxes;
- Euclidean center distance;
- per-track missing-frame count;
- fallen-candidate persistence count;
- bowling trajectory center points.

For a bounding box $b = (x_1, y_1, x_2, y_2)$, its center is computed as:

$$c_x = \frac{x_1 + x_2}{2}, \qquad c_y = \frac{y_1 + y_2}{2}.$$

The center distance between two boxes a and b is:

$$d(a, b) = \sqrt{(c_{x,a} - c_{x,b})^2 + (c_{y,a} - c_{y,b})^2}.$$

The IoU between two boxes is:

$$IoU(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

These engineered geometric features allow the system to maintain object identities across frames without training a separate tracker.

10 Feature Selection

For image detection, feature selection is learned internally by YOLO26n through convolutional and attention-based layers. For tracking, feature selection is rule-based. The test notebook uses only geometry and class identity:

- IoU is selected to preserve spatial overlap continuity;
- center distance is selected to handle objects whose boxes move but still represent the same pin;
- class ID is selected to determine whether a track should be considered standing or fallen;
- confidence is selected to filter weak detections.

This design keeps the tracker lightweight and suitable for video analysis.

11 Data Preprocessing

11.1 Training Preprocessing

The training notebook prepares the bowling gameplay dataset by:

1. downloading the Roboflow YOLO26 export;
2. loading `data.yaml`;
3. validating class names;
4. rewriting bowling gameplay dataset paths;
5. counting labels;
6. visualizing labels;
7. feeding the cleaned configuration into Ultralytics.

During training, Ultralytics resizes images to 640×640, batches them, normalizes pixel values internally, applies training augmentations, and converts labels into the format required for YOLO loss computation.

11.2 Inference Preprocessing

In the test notebook, each video frame is passed to:

```
model.predict(source=frame, imgsz=640, conf=0.10, iou=0.45)
```

Ultralytics handles resizing and normalization internally. Detections are then filtered by class-specific thresholds, converted into NumPy arrays, and passed to the tracking logic.

12 Data Augmentation

The training command did not manually specify a full custom augmentation block, but the logged Ultralytics trainer configuration shows active default augmentations:

- HSV hue, saturation, and value variation;
- translation;

- scaling;
- horizontal flipping;
- mosaic augmentation;
- random erasing;
- Albumentations blur, median blur, grayscale conversion, and CLAHE.

These augmentations are relevant because recorded videos may contain motion blur, lighting variation, camera movement, and object overlap. Mosaic augmentation improves robustness to crowded object layouts, while HSV and blur augmentations improve robustness to appearance variation.

13 Data Splitting

Roboflow provided train, validation, and test splits. The notebook used:

- `train/images` for training;
- `valid/images` for validation;
- `test/images` for testing.

The validation set was used during training to monitor generalization and trigger early stopping. The test set was used after training to estimate final held-out performance.

14 Model Architecture

14.1 YOLO26n Overview

The model used in the training notebook is:

```
model = YOLO("yolo26n.pt")
```

YOLO26n is a compact single-stage detector. It predicts bounding boxes and class probabilities directly from image features. The training log reports:

- 260 layers before fusion;
- 2,505,360 parameters during training;
- 5.8 GFLOPs;
- 4 output classes.

After fusion for validation, the model summary reports:

- 122 layers;
- 2,375,616 parameters;
- 5.2 GFLOPs.

14.2 Architecture Blocks

The printed architecture includes:

- convolutional downsampling layers;
- C3k2 feature extraction blocks;
- SPPF spatial pyramid pooling;

- C2PSA attention-style processing;
- upsampling and concatenation layers for feature fusion;
- a Detect head with feature maps from three scales.

The three-scale detection head is important because pins and the bowling can appear at different sizes depending on camera distance.

14.3 Input and Output Shapes

The exported PyTorch model uses input shape:

$$(1, 3, 640, 640),$$

where the dimensions represent batch size, RGB channels, height, and width. The exported TensorFlow model reports output shape:

$$(1, 300, 6),$$

which corresponds to up to 300 detections, each containing:

$$[x_1, y_1, x_2, y_2, confidence, class_id].$$

15 Model Design Decisions

YOLO26n was selected because the task benefits from a compact detector that can process many video frames efficiently. A larger detector may improve accuracy but would increase latency and memory usage. A nano-scale model provides a practical trade-off between detection quality and computational cost.

Transfer learning was used instead of training from scratch. The log reports that 606 out of 708 pretrained items were transferred. This is beneficial because early convolutional filters already encode general visual patterns such as efficients, corners, textures, and local shapes. Fine-tuning adapts those representations to the specific bowling-bowling classes.

The detector is intentionally paired with a rule-based temporal tracker rather than a heavy neural video model. This keeps the video analysis pipeline simpler and computationally efficient.

16 Training Pipeline

The training call is:

```
model.train(
    data=data_yaml_path,
    epochs=100,
    imgsz=640,
    batch=16,
    device=0,
    workers=2,
    pretrained=True,
    optimizer="AdamW",
    lr0=0.001,
    lrf=0.01,
    weight_decay=0.0005,
    patience=25,
    cos_lr=True,
```

```
    project="/kaggle/working/runs",
    name="rc_bowling_yolo26n_no_extra_aug",
    exist_ok=True
)
```

The training was performed on a Tesla T4 GPU. Automatic Mixed Precision checks passed, allowing faster training and lower GPU memory usage.

16.1 Hyperparameters

Table 2: Main training hyperparameters used in the notebook.

Hyperparameter	Value
Model	YOLO26n
Epochs	100 maximum
Completed epochs	96 due to early stopping
Best epoch	71
Image size	640
Batch size	16
Optimizer	AdamW
Initial learning rate	0.001
Final LR factor	0.01
Weight decay	0.0005
Patience	25
Scheduler	Cosine LR
Workers	2
Device	Tesla T4 GPU
Framework	Ultralytics 8.4.46 / PyTorch 2.10.0

17 Loss Functions

The YOLO training log reports three loss components:

- **Box loss:** localization error between predicted and ground-truth boxes;
- **Class loss:** classification error for object classes;
- **DFL loss:** distribution focal loss used for bounding-box regression precision.

The overall optimization objective can be described as:

$$\mathcal{L}_{total} = \lambda_{box}\mathcal{L}_{box} + \lambda_{cls}\mathcal{L}_{cls} + \lambda_{dfl}\mathcal{L}_{dfl}.$$

During backpropagation, gradients of this total loss update model parameters through AdamW.

18 Optimizer and Learning Rate Strategy

AdamW was used as the optimizer:

$$\theta_{t+1} = \theta_t - \eta \cdot \hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon) - \eta\lambda\theta_t,$$

where the last term represents decoupled weight decay. This optimizer is suitable for deep learning models because it combines adaptive gradient moments with explicit regularization.

Cosine learning-rate scheduling was enabled. This gradually decays the learning rate from the initial value toward the final factor, allowing large updates early and more stable refinement later.

19 Training Procedure

Training progressed for 96 epochs and stopped early because no improvement was observed for 25 epochs. The best checkpoint was saved at epoch 71. The early stopping behavior prevented unnecessary overfitting and reduced wasted computation.

The model improved rapidly in the first ten epochs:

- mAP@0.50 increased from 0.590 at epoch 1 to 0.872 at epoch 10;
- mAP@0.50:0.95 increased from 0.407 to 0.671 over the same interval.

Performance then stabilized gradually, with the best validation result observed around epoch 71.

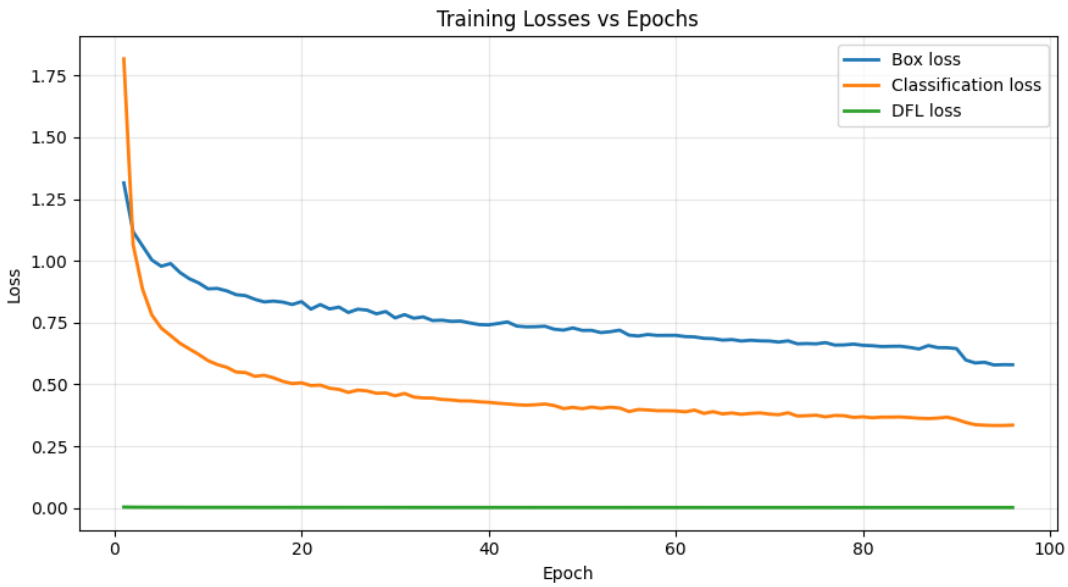

Figure 2: Training and validation loss curves over epochs for the bowling gameplay analysis model.

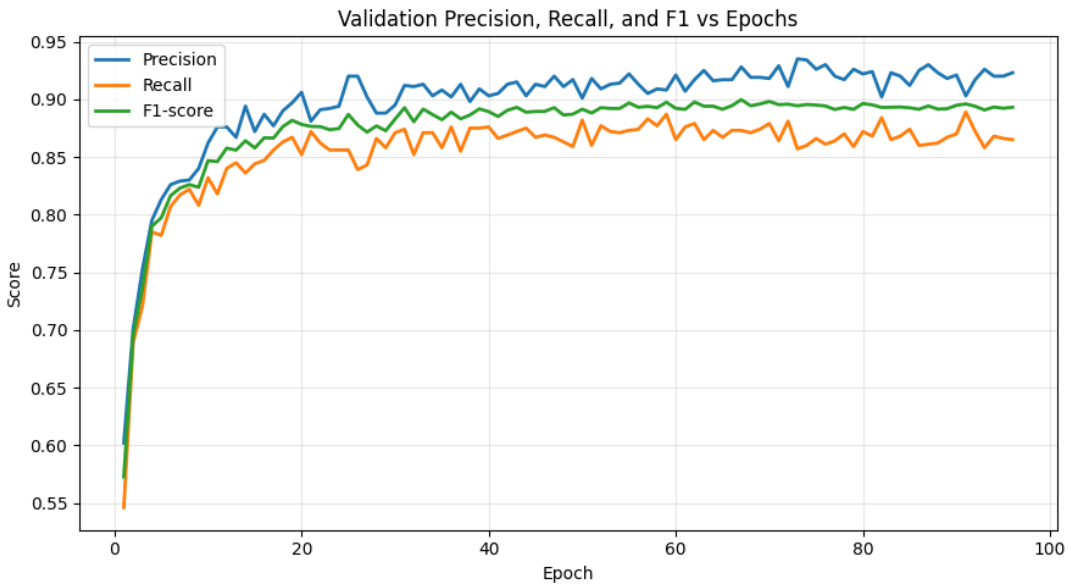

Figure 3: Validation precision, recall, and F1-related performance trends across training epochs.

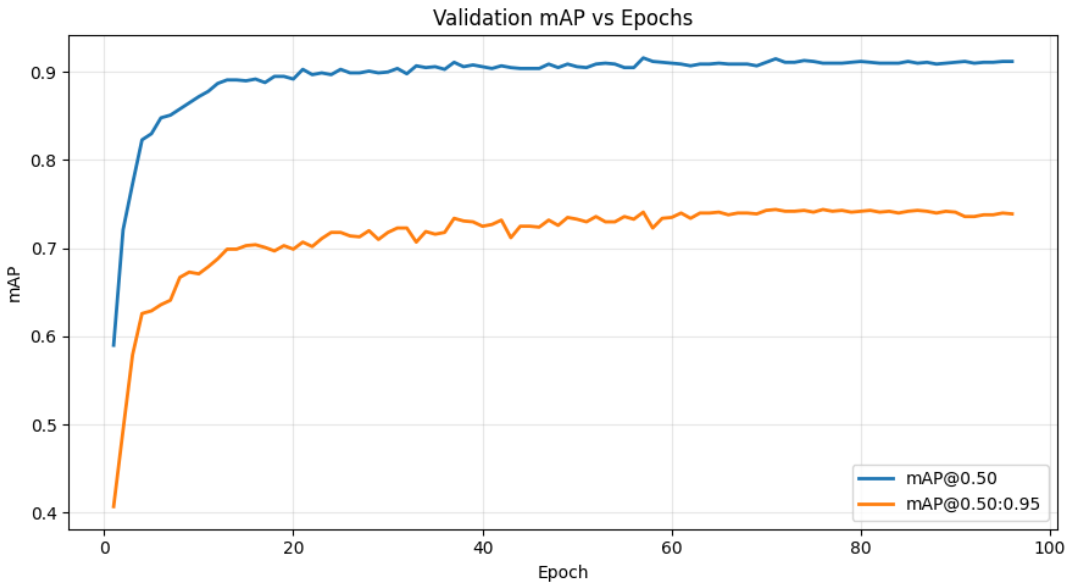

Figure 4: Validation mAP@0.50 and mAP@0.50:0.95 curves for bowling ball and pin detection.

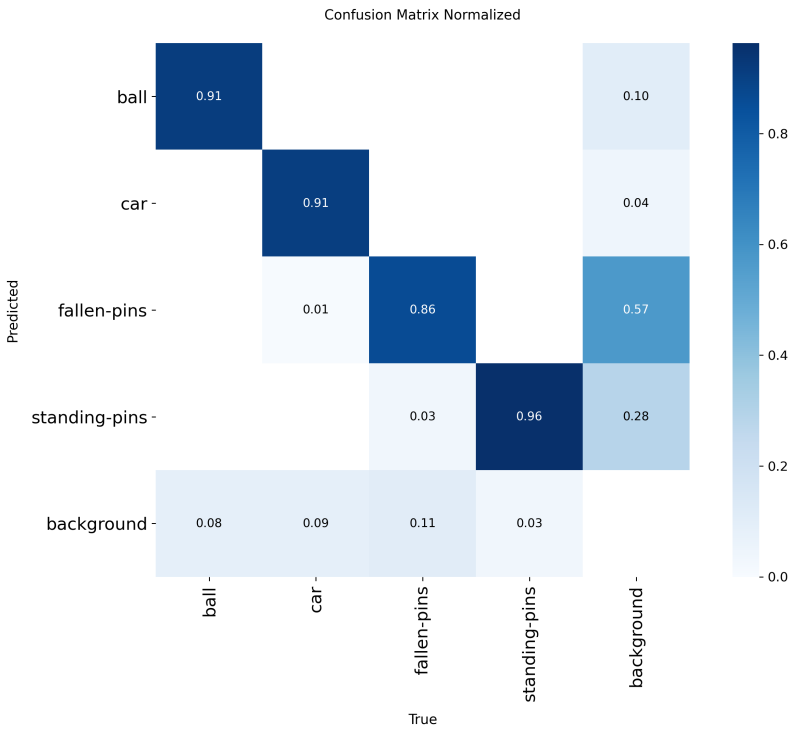

Figure 5: Normalized Confusion matrix for bowling gameplay object detection and pin-state classification.

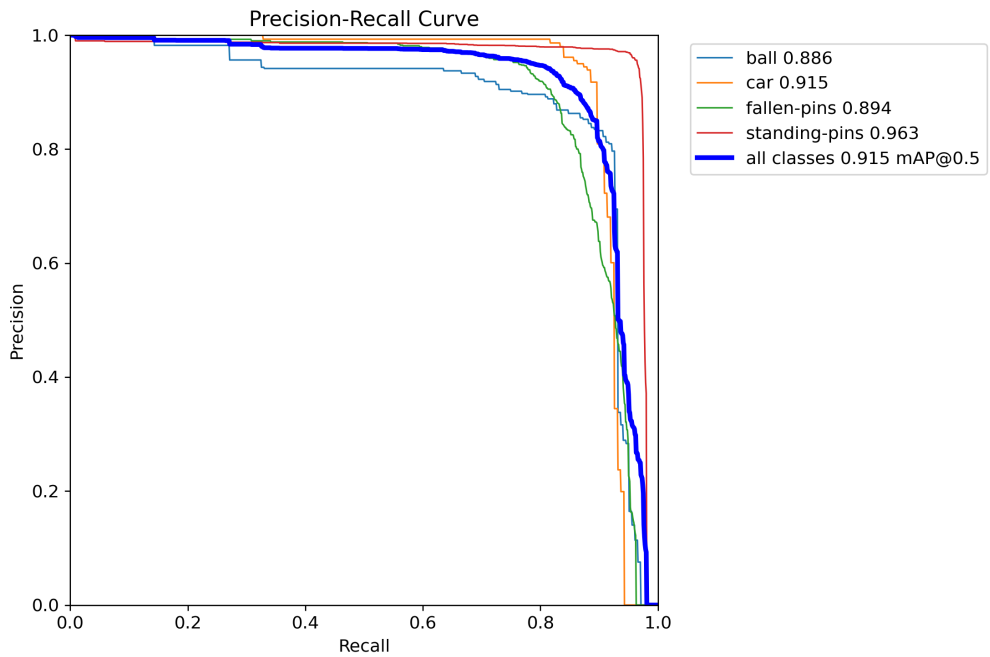

Figure 6: Precision–recall curve for bowling ball and pin detection performance.

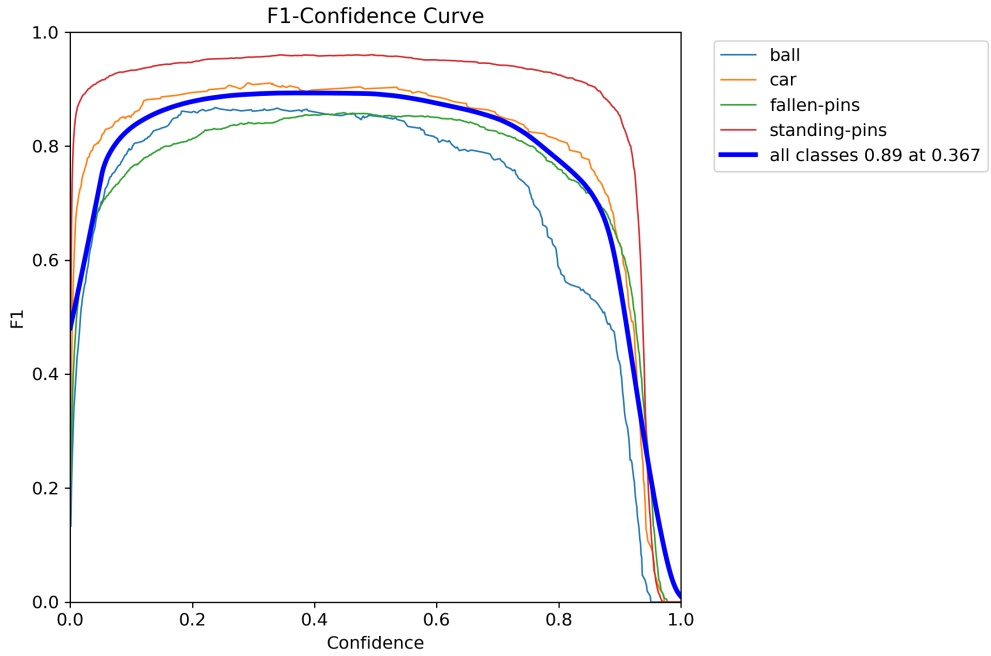

Figure 7: F1-confidence curve showing the effect of confidence threshold selection on bowling gameplay detection performance.

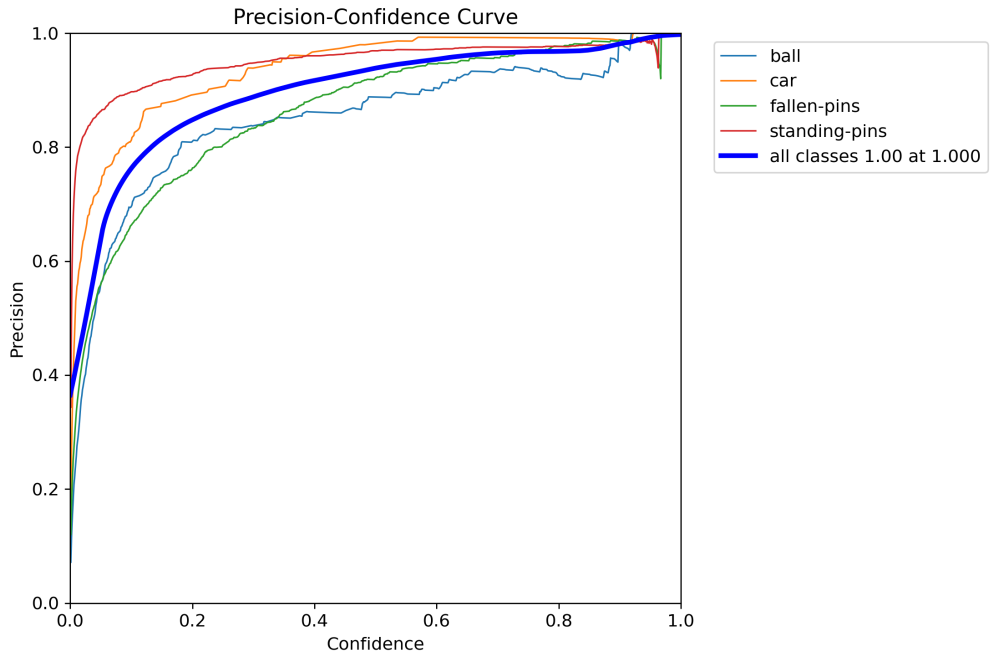

Figure 8: Precision-confidence curve showing precision variation across confidence thresholds for bowling gameplay detection.

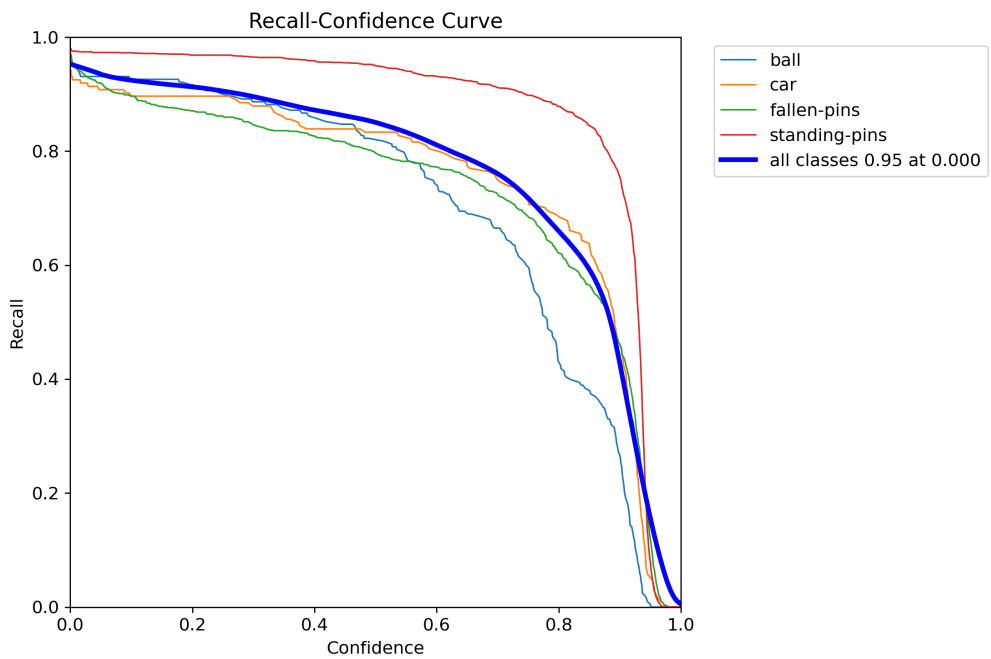

Figure 9: Recall-confidence curve showing recall variation across confidence thresholds for bowling gameplay detection.

20 Validation Logic

Validation was performed automatically after every epoch during training. The notebook also explicitly called:

```
metrics = model.val(
    data=data_yaml_path,
    split="val",
    imgsz=640,
    conf=0.25,
    iou=0.5,
    device=0
)
```

This evaluates predictions against ground-truth boxes using a confidence threshold of 0.25 and IoU threshold of 0.5.

21 Evaluation Metrics

The notebooks report object detection metrics:

- Precision;
- Recall;
- mAP@0.50;
- mAP@0.50:0.95;
- inference speed;
- per-class detection results.

Precision and recall are defined as:

$$Precision = \frac{TP}{TP + FP}, \qquad Recall = \frac{TP}{TP + FN}.$$

The F1-score is computed as:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}.$$

The notebooks did not log classification accuracy or ROC/AUC because YOLO object detection is evaluated primarily through localization-sensitive detection metrics rather than image-level classification metrics. Confusion matrix and curve plots are generated by Ultralytics when `plots=True`, but their numeric matrices are not explicitly printed in the notebook.

22 Experimental Results

22.1 Final Validation Results

Table 3: Final validation metrics for the best YOLO26n checkpoint.

Class	Images	Instances	Precision	Recall	mAP50	mAP50-95
all	240	2983	0.909	0.880	0.915	0.744
ball	129	203	0.853	0.872	0.886	0.637
bowling	167	174	0.961	0.853	0.915	0.707
fallen-pins	210	890	0.867	0.835	0.894	0.749
standing-pins	230	1716	0.957	0.961	0.963	0.882

Standing pins achieved the highest mAP@0.50 because they are visually more consistent and more frequent in the training set. Fallen pins were harder because their appearance changes significantly after impact.

22.2 Held-Out Test Results

Table 4: Held-out test metrics reported by the notebook.

Class	Images	Instances	Precision	Recall	mAP50	mAP50-95
all	120	1336	0.780	0.745	0.762	0.444
ball	63	112	0.701	0.705	0.686	0.320
bowling	77	78	0.681	0.821	0.807	0.418
fallen-pins	104	404	0.788	0.653	0.674	0.431
standing-pins	114	742	0.952	0.800	0.879	0.606

The test results are lower than validation results, showing a realistic generalization gap. The largest difficulty occurs for fallen pins and balls, likely because they appear in more variable positions and can be partially occluded.

22.3 Inference Speed

The best validation run reported:

Table 5: Validation speed reported by Ultralytics.

Stage	Time per image
Preprocessing	0.2 ms
Inference	2.2 ms
Loss	0.0 ms
Postprocessing	2.3 ms

The explicit held-out test run reported 1.9 ms preprocessing, 6.6 ms inference, and 2.6 ms postprocessing per image.

23 Testing Pipeline

The training notebook performs image-level prediction on the test set:

```
model.predict(
    source=TEST_IMAGE_DIR,
    imgsz=640,
    conf=0.20,
    iou=0.45,
    save=True,
    project="/kaggle/working/predictions",
    name="test_predictions",
    exist_ok=True
)
```

This saves annotated prediction images for visual inspection. The output logs show detections for each test image, including the number of balls, fallen pins, and standing pins detected.

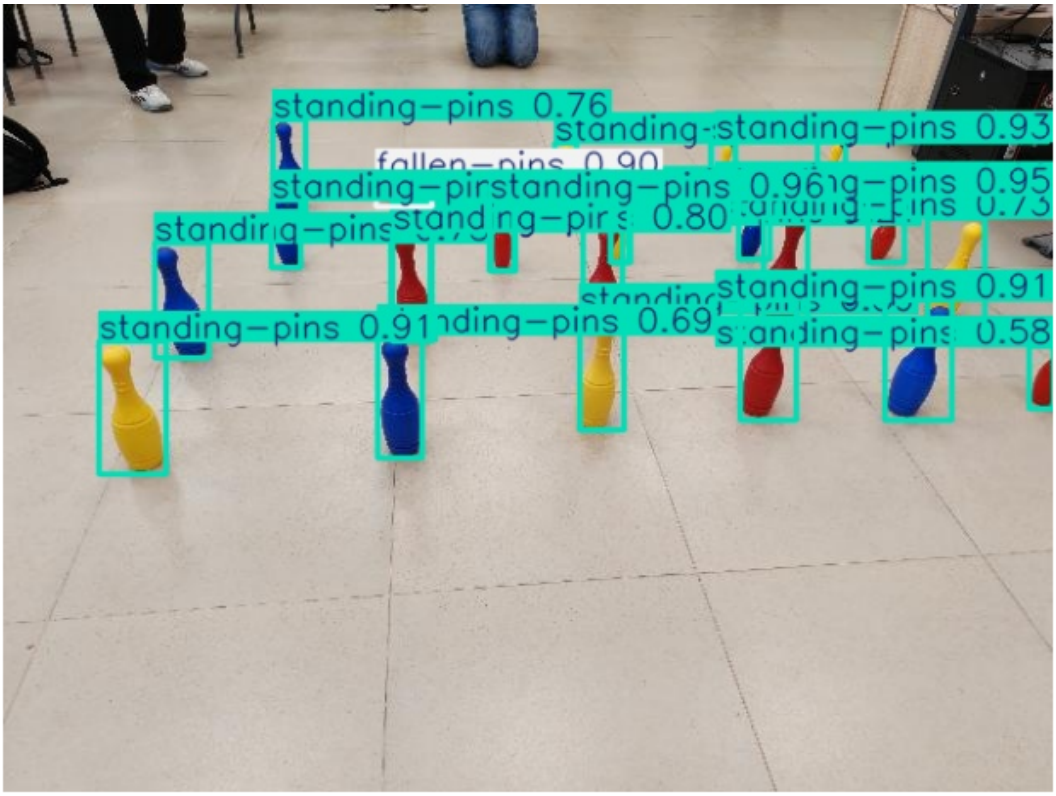

Figure 10: YOLO26n prediction results on bowling gameplay frames showing detected bowling balls and pins.

24 Model Checkpoint Selection and Benchmarking

After training, the best checkpoint was selected automatically using the validation performance tracked by Ultralytics. Early stopping identified epoch 71 as the best epoch, and the corresponding weights were saved as `best.pt`. This checkpoint was then used for image-level test prediction and for the video-level scoring pipeline.

24.1 Benchmark Results

The notebook also reported benchmark measurements for selected runtime formats. These measurements are useful for understanding the computational cost of the detector under the notebook runtime conditions.

Table 6: Selected model benchmark results reported by the notebook.

Format	Status	Size (MB)	mAP50-95	FPS
PyTorch	Success	19.8	0.6013	80.75
TensorRT	Success	11.4	0.6004	314.40
TensorFlow SavedModel	Success	23.5	0.6004	49.37

The benchmark results show that the trained detector is computationally lightweight. The main experimental checkpoint remains the PyTorch `best.pt` model used throughout the reported bowling gameplay inference experiments.

25 Video Inference Pipeline

The notebook `cv-bonus-test.ipynb` implements the full video scoring pipeline. It begins by installing dependencies, importing libraries, finding available videos and models, setting a single input video path, and creating an output directory.

25.1 Model Loading

The video testing notebook sets explicit paths for the trained model and input video. The primary model is loaded through the Ultralytics `YOLO(model_path)` interface. This design allows the same video processing function to be applied to a selected checkpoint while keeping the inference and tracking logic independent from the training code.

25.2 Video Processor

The core function `run_video_scoring`:

1. loads one model using `YOLO(model_path)`;
2. opens the input video with OpenCV;
3. obtains FPS, width, height, and frame count;
4. creates an MP4 writer;
5. processes frames sequentially;
6. runs YOLO prediction per frame;
7. filters detections by confidence and class;
8. updates pin tracks;
9. draws annotations;
10. saves frame-level logs;
11. writes summary JSON and metrics CSV.

25.3 Video Data Flow Diagram

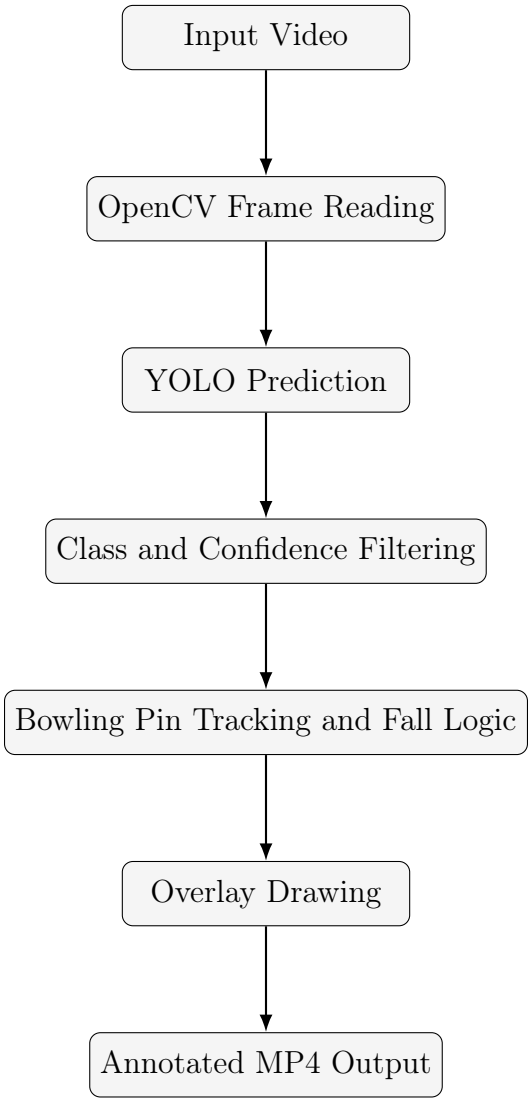

Figure 11: Video inference and scoring pipeline implemented in the testing notebook.

26 Prediction Logic

For each processed frame, the notebook executes:

```
results = model.predict(
    source=frame,
    imgsz=640,
    conf=0.10,
    iou=0.45,
    verbose=False
)[0]
```

Each box is converted into:

- class ID;
- confidence score;
- (x_1, y_1, x_2, y_2) bounding box.

The code then applies per-class confidence thresholds:

Table 7: Class-specific thresholds in the video testing notebook.

Class	Threshold
ball	0.15
bowling	0.15
fallen-pins	0.12
standing-pins	0.12

The lower thresholds improve recall in video frames, where motion blur may reduce confidence. However, they can also increase false positives and duplicate detections, which is why tracking and duplicate-control logic are important.

27 Tracking Logic

27.1 PinTrack Class

The testing notebook defines a `PinTrack` class. Each track stores:

- track ID;
- latest bounding box;
- last seen frame;
- missing-frame count;
- current state;
- last class ID;
- confidence score;
- fallen-candidate count;
- fall order;
- fall frame;
- history.

This class transforms independent detections into persistent temporal objects.

27.2 Matching Strategy

For each existing track, the algorithm searches the current pin detections and computes:

$$score = 2 \cdot IoU - \frac{distance}{CENTER_DISTANCE_THRESHOLD}.$$

A detection can match a track if:

$$IoU \geq 0.10 \quad \text{or} \quad distance \leq 160.$$

The best scoring match updates the track. If no detection matches, the track is marked missing.

27.3 Track Creation and Removal

Unmatched pin detections create new tracks if their confidence is at least 0.12. Tracks are removed if missing for more than 25 frames. This design allows temporary missed detections while preventing old tracks from remaining forever.

27.4 Fall Confirmation

A track becomes confirmed as fallen when its `fallen_candidate_count` reaches 3. This means the same tracked pin must be classified as fallen across multiple updates before a fall order is assigned. This temporal requirement reduces false scoring from single-frame misclassification.

28 Overlay Rendering

The testing notebook uses OpenCV drawing functions to render:

- yellow ball boxes;
- magenta bowling boxes and trajectory;
- green active standing pin tracks;
- red confirmed fallen pin tracks;
- numbered circles for fall order;
- top-left summary panel containing model name, elapsed time, and fallen count.

The output video is written frame by frame through `cv2.VideoWriter` using MP4 encoding.

29 Metrics and Output Files from Video Testing

For every tested model, the video notebook saves:

- annotated video;
- frame-level CSV metrics;
- summary JSON file;
- final comparison CSV.

The summary includes:

- model name and path;
- input video path;
- output path;
- FPS;
- width and height;
- total frames;
- processing runtime;
- processing FPS;
- final fallen count;
- total active tracks;
- number of bowling-path points.

30 Error Analysis

The implementation and notebook outputs reveal several important error sources:

- **Fallen-pin ambiguity:** fallen pins appear in many orientations, leading to lower recall and mAP than standing pins.
- **Duplicate detections:** the test prediction logs show images with many predicted standing pins, which may exceed the number of physical pins in the scene.
- **Class imbalance:** standing pins dominate the bowling gameplay dataset, improving their performance but making minority classes more challenging.
- **Video motion blur:** fast bowling motion and falling pins can reduce detector confidence.
- **Generalization gap:** validation mAP@0.50:0.95 was 0.744, while held-out test mAP@0.50:0.95 was 0.444.

The video notebook addresses some errors through temporal tracking and fall confirmation, but detection quality remains the foundation of the scoring system.

31 Performance Analysis

The YOLO26n detector provides strong speed because it is small and computationally efficient. Validation inference speed was approximately 2.2 ms per image in the best validation run. The benchmark cell reported PyTorch inference at 12.38 ms per image and approximately 80.75 FPS under its benchmark conditions. TensorRT achieved 3.18 ms per image and 314.4 FPS in the notebook benchmark.

The video pipeline adds additional cost beyond single-image inference because it performs frame decoding, detection filtering, geometric matching, overlay drawing, output writing, and metrics logging. Therefore, complete video runtime is determined by both detector speed and the surrounding frame-processing operations.

32 Challenges Faced

The main challenges were:

- **Small-object detection:** pins and balls occupy small image regions.
- **State classification:** fallen and standing pins are visually related but semantically different.
- **Temporal identity:** YOLO detections do not automatically preserve object identity across frames.
- **Duplicate detections:** low inference thresholds improve recall but increase false positives.
- **Video ambiguity:** camera movement, fast bowling motion, and pin overlap introduce frame-level instability.

33 Future Improvements

Future improvements include:

- collecting more test-like videos with varied lighting, camera angles, and pin layouts;
- cleaning duplicate annotations before training;
- adding explicit duplicate suppression in the bowling gameplay inference notebook;
- using stronger temporal fall confirmation rules;
- locking the initial pin count to prevent fake mid-video tracks;
- integrating ByteTrack or BoT-SORT more consistently into the custom scoring layer;
- expanding evaluation with more annotated video-level ground truth.

34 Conclusion

The two notebooks implement a complete object detection and video scoring pipeline for bowling-bowling analysis. The training notebook successfully prepares a Roboflow bowling gameplay dataset, trains a YOLO26n detector, evaluates it, and visualizes predictions. The testing notebook applies trained models to complete videos, tracks pins through time, confirms fall events, draws annotations, and exports result videos with frame-level metrics.

The final detector achieved strong validation performance, especially on standing pins, and the video pipeline demonstrates how frame-level detections can be converted into higher-level scoring events through deterministic temporal logic. The project therefore combines deep learning detection with classical geometric tracking to solve a practical video understanding and scoring problem.

Ultralytics, *YOLO Documentation and Training Framework*. Available: https://docs.ultralytics.com/

Roboflow, *Dataset Management and Annotation Platform*. Available: https://roboflow.com/

OpenCV, *Open Source Computer Vision Library*. Available: https://opencv.org/